

What is MPI ?

Message Passing Interface

A library that allows message exchanges between processes on nodes that can be remote and/or heterogeneous

Initially meant for C, C++, and Fortran
Bindings for Java, MATLAB, Python, ...

Basic Primitives

MPI programs *must start* with the following **initialization** call:
`int MPI_Init(int* argc, char*** argv) ;`

MPI programs *must end* with the following **finalization** call:
`int MPI_Finalize() ;`

For termination purposes, it is imperative that every process calls this *primitive*.

Lesson Plan

Overview

Communicators

Messages

Communication

- Point to point
 - Synchronous / asynchronous
 - Blocking / non-blocking
- Collective

How does one program with MPI?

Every process gets its own flow of control and address space
But all displays are rerouted to the local host of the program

MPI uses its internal representation of data to mask heterogeneity

MPI library calls provide access to communication routines

Easy to identify because they start with `MPI_`

What's in MPI ?

Point-to-point communication routines

- Predefined user operations
- Several communication modes
- Support for structured buffers and types
- Support for heterogeneity

Group communication routines

- Extend point-to-point routines
- Message exchanges among subsets of processes

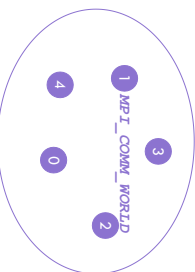
DISTRIBUTED SYSTEMS

Default Communicator

The `MPI_COMM_WORLD` communicator contains all running processes

```
int rank;  
MPI_Comm_rank(MPI_COMM_WORLD,  
               &rank);
```

The pair (`MPI_COMM_WORLD`, `rank`) identifies every process uniquely



Information about the communicator

How many processes belong to communicator `comm` ?

```
int MPI_Comm_size(MPI_Comm comm, int  
                  *size);
```

I'm a process, which `rank` did communicator `comm` assign to me?

```
int MPI_Comm_rank(MPI_Comm comm, int*  
                  rank);
```

Notion of communicator

Type `MPI_Comm` defines a static set of processes Can be generated/destroyed at runtime

Used to define groups for collective communications

Each process gets associated with a unique identifier (`rank`) per communicator

A process can belong to several communicators, possibly with a different rank in each

Each MPI communication goes through a communicator

Allows to identify the sender/recipient

MPI has a predefined communicator that provides a unique ID to every running process:

`MPI_COMM_WORLD`

Contents of an MPI message

The data buffer may contain either:

- an array of values of the same primitive type;
- an array of values of the same complex type;
- or other, really complex structures!

You can define your own types, but...

...heterogeneity makes type definition quite complicated!

Message Tags

MPI uses integer values to specify message control purposes

```
int tag
```

Separates control from data

A process can wait for any message of a given tag

Upon reception of a message, its tag must match the awaited value

It is possible to wait for any message, regardless of its tag

```
MPI_ANY_TAG
```

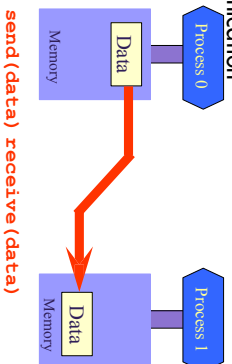
Structure of an MPI message

A message is composed of:

- its application-specific data contents
- address of the buffer
- number of elements
- predefined Data Type (masks heterogeneity)
- its MPI container meta-data
 - the communicator in charge of the exchange
 - the ranks of the processes (source/destination) associated with the message exchange in this communicator
 - a message control tag

Point-to-Point Communication

Simplest mode of communication



`send(buffer, buffer_size, [tag], destination)`
`receive(buffer, buffer_size, [tag], [source])`

MPI Communications

Communications are reliable

Every message sent will be received exactly once

Communications are FIFO

For every couple (P_i, P_j) of processes

And for every couple (m, m') of messages sent by P_i to P_j

If P_i sends m before m', then P_j receives m before m'

This does not apply to messages with different recipients

BEWARE: the reception buffer is not a queue

MPI	C
MPI_CHAR	signed char
MPI_SHORT	signed short
MPI_INT	signed int
MPI_LONG	signed long
MPI_UNSIGNED_CHAR	unsigned char
MPI_UNSIGNED_SHORT	unsigned short
MPI_UNSIGNED_INT	unsigned int
MPI_UNSIGNED_LONG	unsigned long
MPI_FLOAT	float
MPI_DOUBLE	double
MPI_LONG_DOUBLE	long double

MPI Communication Modes

Operation to carry out

Emission of the contents of buffer A by P0

Reception of the data by P1 & storage in buffer B

Options

Communication can be synchronous or asynchronous

Depends on the application environment

Communication can be blocking or non-blocking

Depends on the application protocol

Your First MPI Program

```
#include <mpi.h>
int main(int argc, char
*argv[]) { char msg[20];
int rank;
MPI_Status status;
MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD,
&rank);
if (rank == 0) { /* Only executed by pos 0
*/ strcpy(msg, "Hello world !");
MPI_Send(msg, strlen(msg)+1, MPI_CHAR, 1, 99,
MPI_COMM_WORLD);
}
else {
MPI_Recv(msg, 20, MPI_CHAR, 0, 99, MPI_COMM_WORLD,
&status); printf("I received %s!\n", msg);
}
MPI_Finalize();
}
```

Simplest Subset of MPI Calls

You can write simple MPI programs with only 6 calls!

```
MPI_Init(...)
MPI_Comm_size(...)
MPI_Comm_rank(...)
MPI_Send(...)
MPI_Recv(...)
MPI_Finalize()
```



MPI is simple!

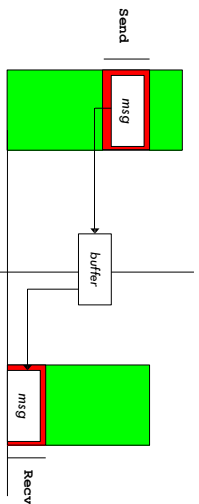
Asynchronous Emission

21

Upon completion of the emission

The data has either been received, or it is in transit

May involve an intermediate memory zone
implementation dependent



Emission Parameters

24

Blocking emission (**Send** and **Ssend**)

- ▢ address of the emission buffer
 - ▢ nb of data items to send
 - ▢ type of all sent data items (homogeneous)
 - ▢ identity of the recipient
 - ▢ message type
 - ▢ communicator
- void* buf
int nb
MPI_Datatype dtype
int dest
int tag
MPI_Comm comm

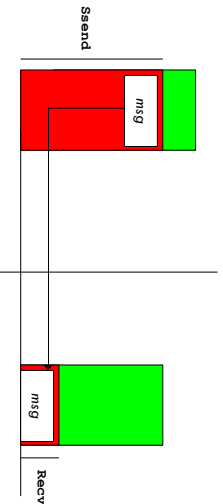
Synchronous Emission

20

A synchronous emission will only be carried out after the recipient calls a reception primitive (RDV)

Does not require an intermediate zone of storage

Data gets copied directly in the reception buffer of the recipient



MPI Calls

23

Emission

- | | |
|-----------|-------------------------|
| MPI_Ssend | synchronous & blocking |
| MPI_Send | asynchronous & blocking |

Reception

- | | |
|----------|-------------------------|
| MPI_Recv | asynchronous & blocking |
|----------|-------------------------|

Blocking Communications

19

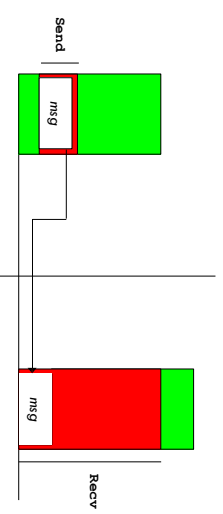
Emission

When the emission is complete, the buffer used to store the sent data can be reused

This does not guarantee reception of the sent data by the recipient (requires synchronous comm.)

Reception

When the reception is complete, the data is available in the buffer of the recipient



Reception

22

A reception is complete when the sent data is available in the reception buffer

Message Status

27

Structure **MPI_Status**

Information available to a recipient

▫ Identity of the sender

`status.MPI_SOURCE`

▫ Message tag

`status.MPI_TAG`

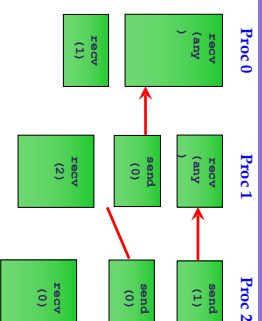
▫ Number of data items sent

`MPI_Get_Count(&status, datatype, &count);`

Message contains count items of type datatype

Beware of Wild Parameters!

30



DEADLOCK!

"Wild" Parameters

26

To receive a message from any sender

`MPI_ANY_SOURCE`

To receive a message of any type

`MPI_ANY_TAG`

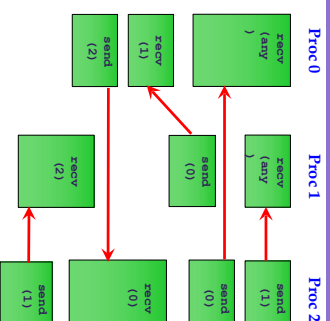
Field `status` provides the means to determine

▫ the identity of the sender

▫ the type of a received message

Beware of Wild Parameters!

32



Execution terminates

Reception Parameters

25

Blocking Reception (**Recv**)

▫ address of the reception buffer

▫ nb of expected data items

▫ Type of all received data items (homogeneous)

▫ Identity of the sender

▫ message type

▫ communicator

▫ additional information

```

void* buf
int nb
MPI_Datatype dtype
int source
int tag
MPI_Comm comm
MPI_Status *status
  
```

Example With 3 Processes

28

```

int main(int argc, char
*argv[]) { int msg = 2;
int rank;
MPI_Status status;

MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD,
&rank);

if (rank == 0) { /* 0 expects 2 messages */
    MPI_Recv(&msg, 1, MPI_INT, MPI_ANY_SOURCE, 99,
MPI_COMM_WORLD, &status);
    printf("Hello %d \n",
status.MPI_SOURCE);
    MPI_Recv(&msg, 1, MPI_INT, MPI_ANY_SOURCE,
99,
MPI_COMM_WORLD, &status);
    printf("Hello %d \n", status.MPI_SOURCE);
} else {
    MPI_Send(&msg, 1, MPI_INT, 0, 99,
MPI_COMM_WORLD);
  
```

A Buffer is NOT a strict Queue!

```
31 int main(int argc, char *argv[]) {
    int msg = 3;
    int rank;
    MPI_Status status;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    if (rank == 0) { /* P0 sends to P1 */
        MPI_Send(&msg, 1, MPI_INT, 1, 98, MPI_COMM_WORLD);
        msg = 5;
        MPI_Send(&msg, 1, MPI_INT, 1, 98, MPI_COMM_WORLD);
    } else {
        MPI_Recv(&msg, 1, MPI_INT, 0, 98, MPI_COMM_WORLD, &status);
        MPI_Recv(&msg, 1, MPI_INT, 0, 98, MPI_COMM_WORLD, &status);
        printf("I received %d\n", msg);
    }
    MPI_Finalize();
    return 0;
}
```

Every run of this other application also terminates
Only one printf call; always displays 'I received 5'

A Buffer is NOT a strict Queue!

```
32 int main(int argc, char *argv[]) {
    int msg = 3;
    int rank;
    MPI_Status status;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    if (rank == 0) { /* P0 sends to P1 */
        MPI_Send(&msg, 1, MPI_INT, 1, 98, MPI_COMM_WORLD);
        msg = 5;
        MPI_Send(&msg, 1, MPI_INT, 1, 99, MPI_COMM_WORLD);
    } else {
        MPI_Recv(&msg, 1, MPI_INT, 0, 99, MPI_COMM_WORLD, &status);
        printf("I received %d\n", msg);
        MPI_Recv(&msg, 1, MPI_INT, 0, 98, MPI_COMM_WORLD, &status);
        printf("I received %d\n", msg);
    }
    MPI_Finalize();
    return 0;
}
```

Every run of this application terminates
Always displays 'I received 5' first, and then 'I received 3'

Testing the Contents of a Message

MPI_Probe(int source,
int tag,
MPI_Comm com,
MPI_Status *status)

Inquires about messages without extracting from the reception queue

Inquiry helps

- Track the arrival of messages from a specific sender and/or with a specific tag
- Prepare an appropriate storage buffer for the received data

Broadcast Example

```
36 #include <stdio.h>
#include
<string.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
    char msg[20];
    int rank;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD,
    &rank);
    if (rank == 0) { /*-- P0 is the broadcaster
        --*/ strcpy(msg, "Hello world !");
    }

    MPI_Bcast(msg, 20, MPI_CHAR, 0, MPI_COMM_WORLD);
    printf("I am %d and I received - %s\n", rank,
    msg); MPI_Finalize();
    return 0;
}
```

Message Broadcast

```
33 int MPI_Bcast(void* buf,
    int count,
    MPI_Datatype
    dtype, int root,
    MPI_Comm comm)
```

The data content of buffer **buf** is sent by process **root** (rank value) to all processes in the communicator

Data content is copied in the buffer **buf** of all the processes in **comm**, including **root**

Group Communications

Principles

High-level routines to handle multiple concurrent communications. Must be called by every process in the communicator

Built on top of point-to-point routines

Calls

- `MPI_Bcast`
Broadcast to every process in the communicator
- `MPI_Barrier`
Synchronization barrier
Every process in the group blocks until all the others reach the barrier point
- `MPI_Scatter`
Distributes distinct data items from a single source to each process in the group
- `MPI_Gather`
Reverse operation of `MPI_Scatter`
- `MPI_Reduce`
Applies a reduction operation on data from all processes in the group and places the result at one destination

Conclusion

37

Programming is very intuitive

Once the algorithm is figured out

Every process is sequential

The programmer is in charge of handling non-determinism
eg. is the next step of the process to send or to receive?

**Enables real distributed execution of processes
on remote nodes**